# 04. Quote 검증 프로세스

## 검증 개요

Quote 검증은 TEE 환경에서 생성된 증명서의 유효성과 플랫폼의 신뢰성을 확인하는 핵심 프로세스입니다. `verify()` 함수를 통해 암호학적 검증, 인증서 체인 확인, TCB 상태 평가 등이 수행됩니다.

## 메인 검증 함수

### verify() 함수 시그니처

```
pub fn verify(
raw_quote: &[u8], // 원본 Quote 바이너리 데이터
collateral: &QuoteCollateralV3, // Collateral 정보
now_secs: u64, // 현재 시간 (Unix timestamp)
) -> Result<VerifiedReport> // 검증 결과
```

### VerifiedReport 구조

```
pub struct VerifiedReport {
pub status: String, // TCB 상태 ("UpToDate", "OutOfDate", etc.)
pub advisory_ids: Vec<String>, // 보안 권고사항 ID 목록
pub report: Report, // 파싱된 Report 데이터
```

```
}
```

## 검증 프로세스 단계별 분석

### 1. 초기 데이터 파싱 및 검증

```
pub fn verify(raw_quote: &[u8], collateral: &QuoteCollateralV3, now_secs: u64)
// Quote 파싱
let mut quote = raw_quote;
let quote = Quote::decode(&mut quote).context("Failed to decode quote")?;
let signed_quote_len = quote.signed_length();

// TCB 정보 파싱
let tcb_info = serde_json::from_str::<TcbInfo>(&collateral.tcb_info)
.context("Failed to decode TcbInfo")?;

// TCB 정보 만료 확인
let next_update = chrono::DateTime::parse_from_rfc3339(&tcb_info.next_update)
.ok()
.context("Failed to parse next update")?;
if now_secs > next_update.timestamp() as u64 {
bail!("TCBInfo expired");
}
```

```
// ...
}
```

## 2. CRL 검증

```
// CRL 배열 구성
let crls = [&collateral.root_ca_crl[..], &collateral.pck_crl];
let now = UnixTime::since_unix_epoch(Duration::from_secs(now_secs));

// 루트 CA CRL 검증 (포크된 webpki 사용)
dcap_qvl_webpki::check_single_cert_crl(TRUSTED_ROOT_CA_DER, &crls, now)?;
```

## 3. Quote 기본 필드 검증

```
// 지원되는 Quote 버전 확인
if ![3, 4, 5].contains(&quote.header.version) {
return Err(anyhow!("Unsupported DCAP quote version"));
}

// 증명 키 타입 확인 (ECDSA P-256만 지원)
if quote.header.attestation_key_type != ATTESTATION_KEY_TYPE_ECDSA256_WITH_P256
bail!("Unsupported DCAP attestation key type");
```

```
}
```

## 4. TCB 정보 인증서 체인 및 서명 검증

```
// TCB 정보 발급자 인증서 체인 추출
let  tcb_leaf_certs = extract_certs(collateral.tcb_info_issuer_chain.as_bytes())
if  tcb_leaf_certs.len() < 2 {
bail!("Certificate chain is too short in quote_collateral");
}

// TCB 리프 인증서 파싱
let  tcb_leaf_cert = webpki::EndEntityCert::try_from(&tcb_leaf_certs[0])
.context("Failed to parse leaf certificate in quote_collateral")?;

// 인증서 체인 검증
verify_certificate_chain(&tcb_leaf_cert, &tcb_leaf_certs[1..], now, &crls)?;

// TCB 정보 서명 검증
let  asn1_signature = encode_as_der(&collateral.tcb_info_signature)?;
if  tcb_leaf_cert
.verify_signature(
webpki::ring::ECDSA_P256_SHA256,
collateral.tcb_info.as_bytes(),
```

```
&asn1_signature,
)
.is_err()
{
return Err(anyhow!("Rsa signature is invalid for tcb_info in quote_collateral"))
}
```

## 5. PCK 인증서 체인 검증

```
// AuthData 추출 및 정규화
let auth_data = quote.auth_data.into_v3();
let certification_data = auth_data.certification_data;

// PCK 인증서 체인 타입 확인
if certification_data.cert_type != PCK_CERT_CHAIN { // 5
bail!("Unsupported DCAP PCK cert format");
}

// PCK 인증서 체인 추출
let qe_certification_certs = extract_certs(&certification_data.body.data)
.context("Failed to extract PCK certificates")?;

if qe_certification_certs.len() < 2 {
```

```
bail!("Certificate chain is too short in quote");
}

// PCK 리프 인증서 파싱 및 체인 검증
let qe_leaf_cert = webpki::EndEntityCert::try_from(&qe_certification_certs[0])
.context("Failed to parse PCK certificate")?;
verify_certificate_chain(&qe_leaf_cert, &qe_certification_certs[1..], now, &crls
```

## 6. QE Report 서명 검증

```
// QE Report 서명 검증
let asn1_signature = encode_as_der(&auth_data.qe_report_signature)?;
if qe_leaf_cert
.verify_signature(
webpki::ring::ECDSA_P256_SHA256,
&auth_data.qe_report,
&asn1_signature,
)
.is_err()
{
return Err(anyhow!("Rsa signature is invalid for qe_report in quote"));
}
```

## 7. QE Report 해시 검증

```
// QE Report 파싱
let mut qe_report = auth_data.qe_report.as_slice();
let qe_report = EnclaveReport::decode(&mut qe_report).context("Failed to decod

// QE 해시 데이터 구성
let mut qe_hash_data = [0u8; QE_HASH_DATA_BYTE_LEN]; // 96바이트
qe_hash_data[0..ATTESTATION_KEY_LEN].copy_from_slice(&auth_data.ecdsa_attestatio
qe_hash_data[ATTESTATION_KEY_LEN..].copy_from_slice(&auth_data.qe_auth_data.data

// SHA-256 해시 계산 및 비교
let qe_hash = ring::digest::digest(&ring::digest::SHA256, &qe_hash_data);
if qe_hash.as_ref() != &qe_report.report_data[0..32] {
bail!("QE report hash mismatch");
}
```

## 8. Quote 서명 검증

```
// ECDSA 공개키 구성 (압축되지 않은 형식)
let mut pub_key = [0x04u8; 65]; // 0x04 접두사 + 64바이트 키
pub_key[1..].copy_from_slice(&auth_data.ecdsa_attestation_key);
```

```
// 공개키 객체 생성
let peer_public_key = ring::signature::UnparsedPublicKey::new(
&ring::signature::ECDSA_P256_SHA256_FIXED,
pub_key
);


// Quote 서명 검증 (헤더 + 리포트 부분만)
peer_public_key
.verify(
raw_quote
.get(..signed_quote_len)
.ok_or(anyhow!("Failed to get signed quote"))?,
&auth_data.ecdsa_signature,
)
.map_err(|_| anyhow!("Isv enclave report signature is invalid"))?;
```

# PCK 인증서 정보 추출

## Intel 확장 정보 추출

```
// PCK 인증서에서 Intel 확장 정보 추출
let extension_section = utils::get_intel_extension(&qe_certification_certs[0])?
let cpu_svn = utils::get_cpu_svn(&extension_section)?; // CPU SVN
```

```
let pce_svn = utils::get_pce_svn(&extension_section)?; // PCE SVN
let fmspc = utils::get_fmspc(&extension_section)?; // FMSPC


// TCB 정보의 FMSPC와 비교
let tcb_fmspc = hex::decode(&tcb_info.fmspc)
.ok()
.context("Failed to decode TCB FMSPC")?;
if fmspc != tcb_fmspc[..] {
bail!("Fmspc mismatch");
}
```

## TCB 상태 평가

### TDX Quote 특별 검증

```
// TDX Quote의 경우 TCB 정보 버전 및 ID 확인
if quote.header.tee_type == TEE_TYPE_TDX && (tcb_info.version < 3 || tcb_info.i
bail!("TDX quote with non-TDX TCB info in the collateral");
}
```

### TCB 레벨 매칭 알고리즘

```
let mut tcb_status = "Unknown".to_owned();
```

```
let mut advisory_ids = Vec::<String>::new();

for tcb_level in &tcb_info.tcb_levels {
// PCE SVN 확인
if pce_svn < tcb_level.tcb.pce_svn {
continue;
}

// SGX 컴포넌트 SVN 확인
let sgx_components = tcb_level
.tcb
.sgx_components
.iter()
.map(|c| c.svn)
.collect::<Vec<_>>();
if sgx_components.is_empty() {
bail!("No SGX components in the TCB info");
}
if cpu_svn[..] < sgx_components[..] {
continue;
}

// TDX의 경우 추가 컴포넌트 확인
```

```
if  quote.header.tee_type == TEE_TYPE_TDX {
let  td_report = quote
.report
.as_td10()
.context("Failed to get TD10 report")?;
let  tdx_components = tcb_level
.tcb
.tdx_components
.iter()
.map(|c| c.svn)
.collect::<Vec<_>>();
if  tdx_components.is_empty() {
bail!("No TDX components in the TCB info");
}
if  td_report.tee_tcb_svn[..] < tdx_components[..] {
continue;
}
}


// 매칭되는 TCB 레벨 발견
tcb_status = tcb_level.tcb_status.clone();
tcb_level
.advisory_ids
```

```
.iter()
.for_each(|id| advisory_ids.push(id.clone()));
break;
}
```

## 속성 검증

### validate_attrs() 함수

```
fn validate_attrs(report: &Report) -> Result<()> {
match &report {
Report::TD15(report) => validate_td15(report),
Report::TD10(report) => validate_td10(report),
Report::SgxEnclave(report) => validate_sgx(report),
}
}
```

### SGX 속성 검증

```
fn validate_sgx(report: &EnclaveReport) -> Result<()> {
let is_debug = report.attributes[0] & 0x02 != 0; // DEBUG 비트 확인
if is_debug {
bail!("Debug mode is enabled");
```

```
}
Ok(())
}
```

## TDX 속성 검증 (TD10)

```
fn validate_td10(report: &TDReport10) -> Result<()> {
let td_attrs = TDAttributes::parse(report.td_attributes)
.context("Failed to parse TD attributes")?;

// 디버그 모드 확인
if td_attrs.tud != 0 {
bail!("Debug mode is enabled");
}

// 예약 비트 확인
if td_attrs.sec.reserved_lower != 0
|| td_attrs.sec.reserved_bit29
|| td_attrs.other.reserved != 0
{
bail!("Reserved bits in TD attributes are set");
}
```

```
// 보안 속성 확인
if !td_attrs.sec.sept_ve_disable {
bail!("SEPT_VE_DISABLE is not enabled");
}
if td_attrs.sec.pks {
bail!("PKS is enabled");
}
if td_attrs.sec.kl {
bail!("KL is enabled");
}

Ok(())
}
```

## TDX 속성 검증 (TD15)

```
fn validate_td15(report: &TDReport15) -> Result<()> {
// 서비스 TD 측정값 확인 (모두 0이어야 함)
if report.mr_service_td != [0u8; 48] {
bail!("Invalid mr service td");
}

// 기본 TD10 검증 수행
```

```
validate_td10(&report.base)

}
```

## 유틸리티 함수들

### DER 인코딩 함수

```
pub fn encode_as_der(data: &[u8]) -> Result<Vec<u8>> {

if data.len() != 64 { // ECDSA P-256 서명은 64바이트

bail!("Key length is invalid");

}


let mut sequence = der::asn1::SequenceOf::<der::asn1::UintRef, 2>::new();

sequence.add(der::asn1::UintRef::new(&data[0..32])?)?; // r 값

sequence.add(der::asn1::UintRef::new(&data[32..])?)?; // s 값


let mut asn1 = alloc::vec![0u8; 72]; // 충분한 버퍼 크기

let mut writer = der::SliceWriter::new(&mut asn1);

writer.encode(&sequence)?;

Ok(writer.finish()?.to_vec())

}
```

### 인증서 체인 검증 함수

```rust
pub fn verify_certificate_chain(
leaf_cert: &webpki::EndEntityCert,
intermediate_certs: &[CertificateDer],
time: UnixTime,
crl_der: &[&[u8]],
) -> Result<()> {
let sig_algs = webpki::ALL_VERIFICATION_ALGS;


// CRL 파싱
let crls: Vec<CertRevocationList> = crl_der
.iter()
.map(|der| BorrowedCertRevocationList::from_der(der).map(|crl| crl.into()))
.collect::<Result<Vec<_>, _>>()?;
let crl_slice = crls.iter().collect::<Vec<_>>();


// 폐기 옵션 구성
let builder = webpki::RevocationOptionsBuilder::new(&crl_slice)?;
let revocation = builder
.with_depth(webpki::RevocationCheckDepth::Chain) // 전체 체인 검사
.with_status_policy(webpki::UnknownStatusPolicy::Deny) // 알 수 없는 상태 거부
.with_expiration_policy(webpki::ExpirationPolicy::Enforce) // 만료 정책 강제
.build();
```

```
// 인증서 체인 검증
leaf_cert
.verify_for_usage(
sig_algs,
&[sgx_pck_root()], // Intel SGX 루트 CA
intermediate_certs,
time,
webpki::KeyUsage::server_auth(),
Some(revocation),
None,
)
.context("Failed to verify certificate chain")?;


Ok(())
}
```

## 최종 결과 반환

### VerifiedReport 생성

```
validate_attrs(&quote.report)?; // 최종 속성 검증
```

```
Ok(VerifiedReport {

status: tcb_status, // "UpToDate", "OutOfDate", "ConfigurationNeeded" 등

advisory_ids, // 보안 권고사항 ID 목록

report: quote.report, // 파싱된 Report 데이터

})
```

이러한 단계별 검증 프로세스를 통해 Quote의 암호학적 무결성, 인증서 체인의 유효성, 플랫폼의 TCB 상태가 종합적으로 평가됩니다.